# GAZİ UNIVERSITY

# FACULTY OF ENGINEERING
# DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 12: JOINS

# JOINS

INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN

# JOINS

PostgreSQL join is used to combine columns from one (self-join) or more tables based on the values of the common columns between related tables. The common columns are typically the primary key columns of the first table and the foreign key columns of the second table.

PostgreSQL supports inner join, left join, right join, full outer join, cross join, natural join, and a special kind of join called self-join.

# JOINS

Suppose you have two tables called teams and players:

```sql
CREATE TABLE teams (
    id INT PRIMARY KEY,
    team VARCHAR (100) NOT NULL,
    city VARCHAR (100) NOT NULL
);

CREATE TABLE players (
    id INT PRIMARY KEY,
    team_id INT REFERENCES teams (id),
    player VARCHAR (100) NOT NULL,
    role VARCHAR (100) NOT NULL
);
```

```sql
INSERT INTO teams (id, team, city)
VALUES
    (1, 'Lions', 'Rome'),
    (2, 'Owls', 'Oslo'),
    (3, 'Bears', 'Bern'),
    (4, 'Sharks', 'Lima');

INSERT INTO players (id, team_id, player, role)
VALUES
    (1, 1, 'Ava', 'Guard'),
    (2, 1, 'Noah', 'Wing'),
    (3, 2, 'Emma', 'Back'),
    (4, NULL, 'Liam', 'Guard'),
    (5, NULL, 'Mia', 'Wing');
```

A team can have many players. Some players may not belong to a team yet, so their team_id is NULL.

# JOINS

Currently we have two tables with following values:

```
SELECT * FROM teams;
```

```
id |  team   | city
----+--------+------
  1 | Lions   | Rome
  2 | Owls    | Oslo
  3 | Bears   | Bern
  4 | Sharks | Lima
(4 rows)
```

```
SELECT * FROM players;
```

```
id | team_id | player | role
----+---------+--------+-------
  1 |       1 | Ava    | Guard
  2 |       1 | Noah   | Wing
  3 |       2 | Emma   | Back
  4 |    null | Liam   | Guard
  5 |    null | Mia    | Wing
(5 rows)
```

# INNER JOIN

The following statement joins the first table (`teams`) with the second table (`players`) by matching the values in the `id` and `team_id` columns:

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
INNER JOIN players
    ON teams.id = players.team_id;
```

# INNER JOIN

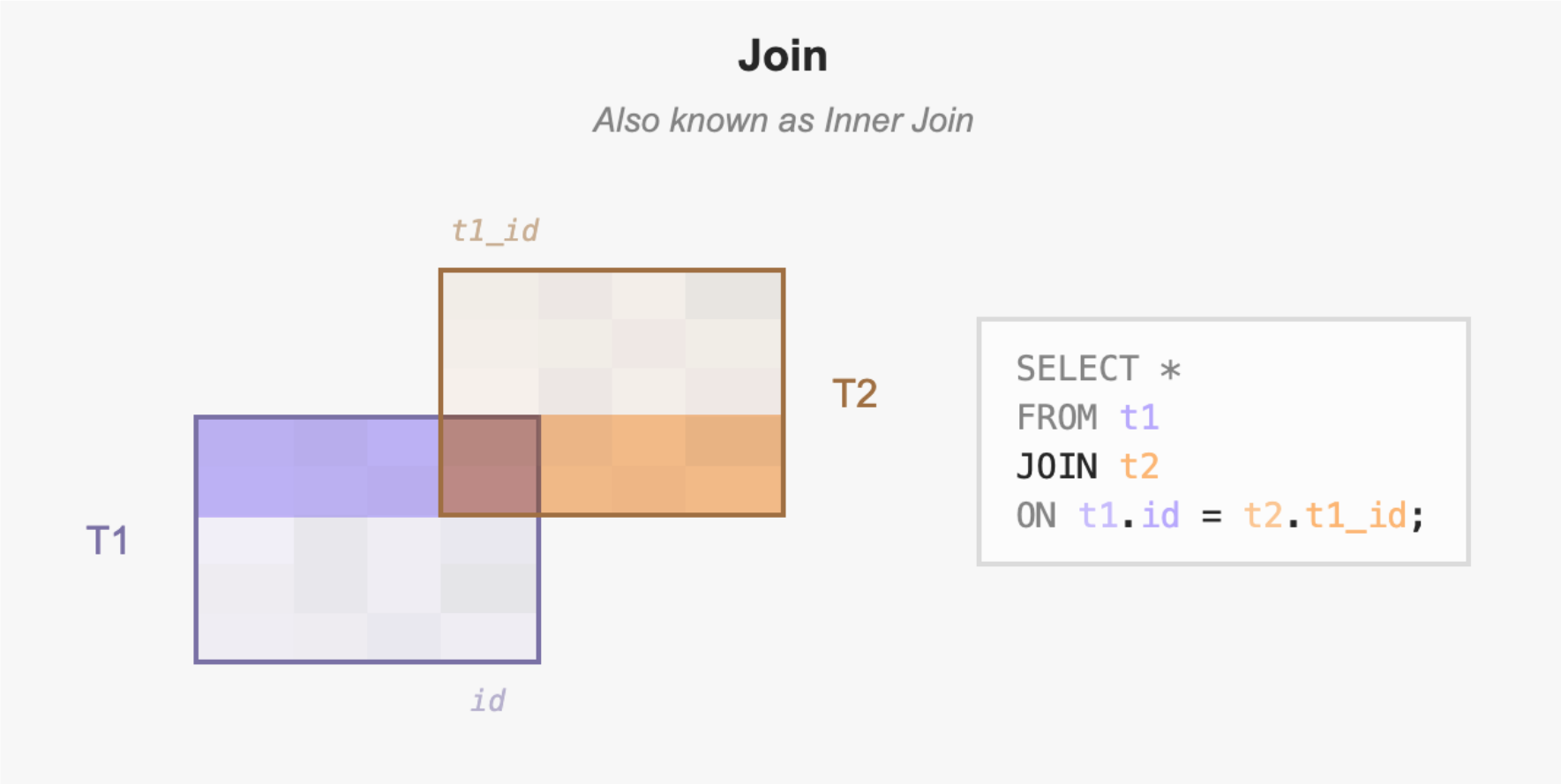
It is common practice to use `JOIN` as shorthand for `INNER JOIN`. Output:

```
team_id | team  | city | player_id | player | role
---------+-------+------+-----------+--------+-------
       1 | Lions | Rome |         1 | Ava    | Guard
       1 | Lions | Rome |         2 | Noah   | Wing
       2 | Owls  | Oslo |         3 | Emma   | Back
(3 rows)
```

The inner join examines each row in the first table (`teams`). It compares the value in the `id` column with the value in the `team_id` column of each row in the second table (`players`). If these values are equal, the inner join creates a new row that contains columns from both tables and adds this new row to the result set.

# INNER JOIN

The following diagram illustrates the inner join:




NOTE: This Venn diagram is a conceptual illustration of the join operation. It does not depict individual row-level matching or Cartesian products.

# LEFT JOIN

The following statement uses the left join clause to join the `teams` table with the `players` table. In the left join context, the first table is called the left table and the second table is called the right table.

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
LEFT JOIN players
   ON teams.id = players.team_id;
```

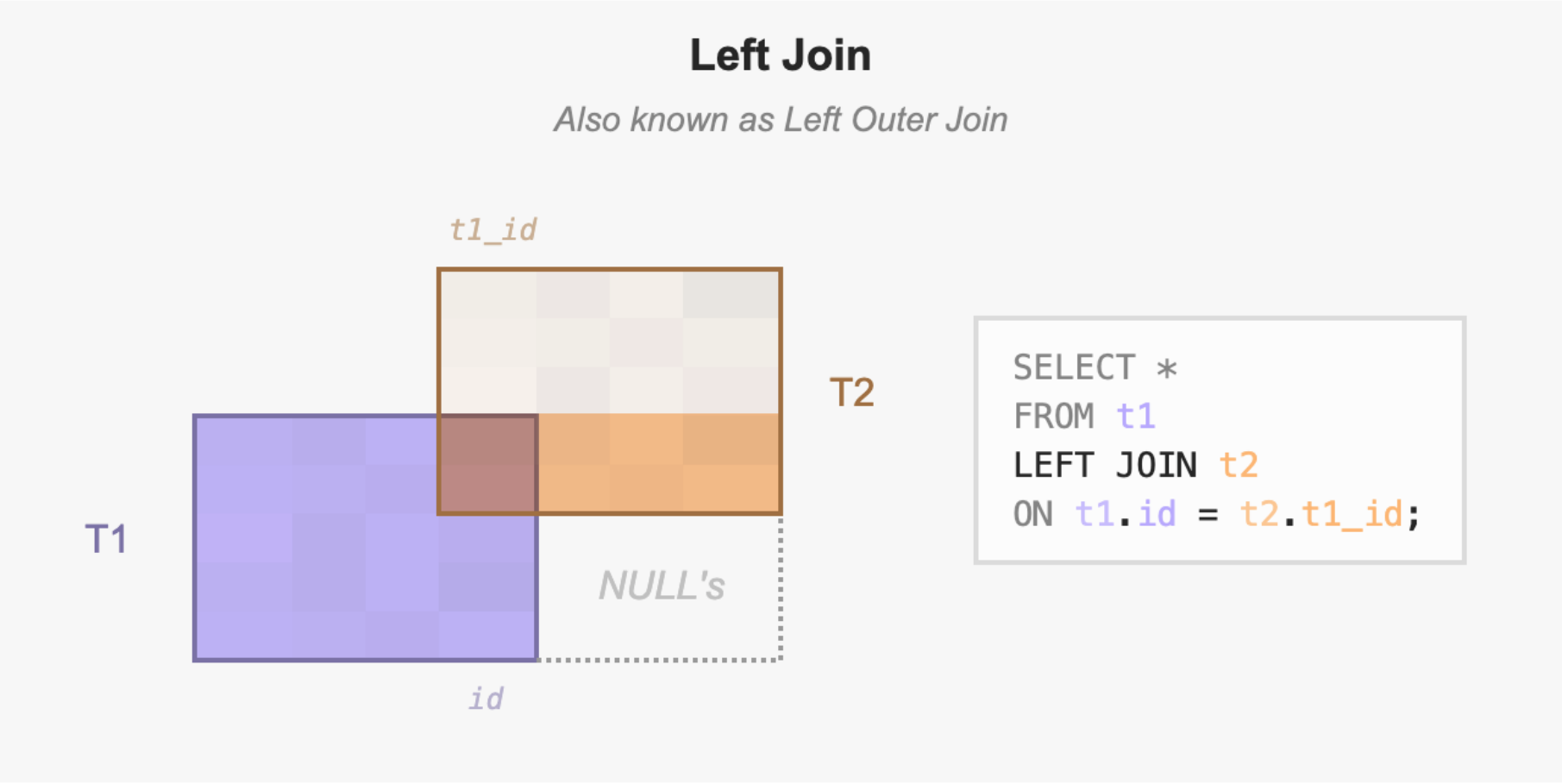
```
team_id |  team   | city | player_id | player | role
--------+--------+------+-----------+--------+-------
      1 | Lions   | Rome |         1 | Ava    | Guard
      1 | Lions   | Rome |         2 | Noah   | Wing
      2 | Owls    | Oslo |         3 | Emma   | Back
      3 | Bears   | Bern |      null | null   | null
      4 | Sharks | Lima |      null | null   | null
(5 rows)
```

# LEFT JOIN

- The left join starts selecting data from the left table. It compares values in the `id` column with the values in the `team_id` column in the `players` table.
- If these values are equal, the left join creates a new row that contains columns of both tables and adds this new row to the result set. (see the first three rows in the result set).
- In case the values do not equal, the left join also creates a new row that contains columns from both tables and adds it to the result set. However, it fills the columns of the right table (`players`) with null. (see the last two rows in the result set).

# LEFT JOIN

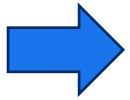
The following diagram illustrates the left join:



# LEFT JOIN

To select rows from the left table that do not have matching rows in the right table, you use the left join with a `WHERE` clause. For example:

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
LEFT JOIN players
    ON teams.id = players.team_id
WHERE players.id IS NULL;
```



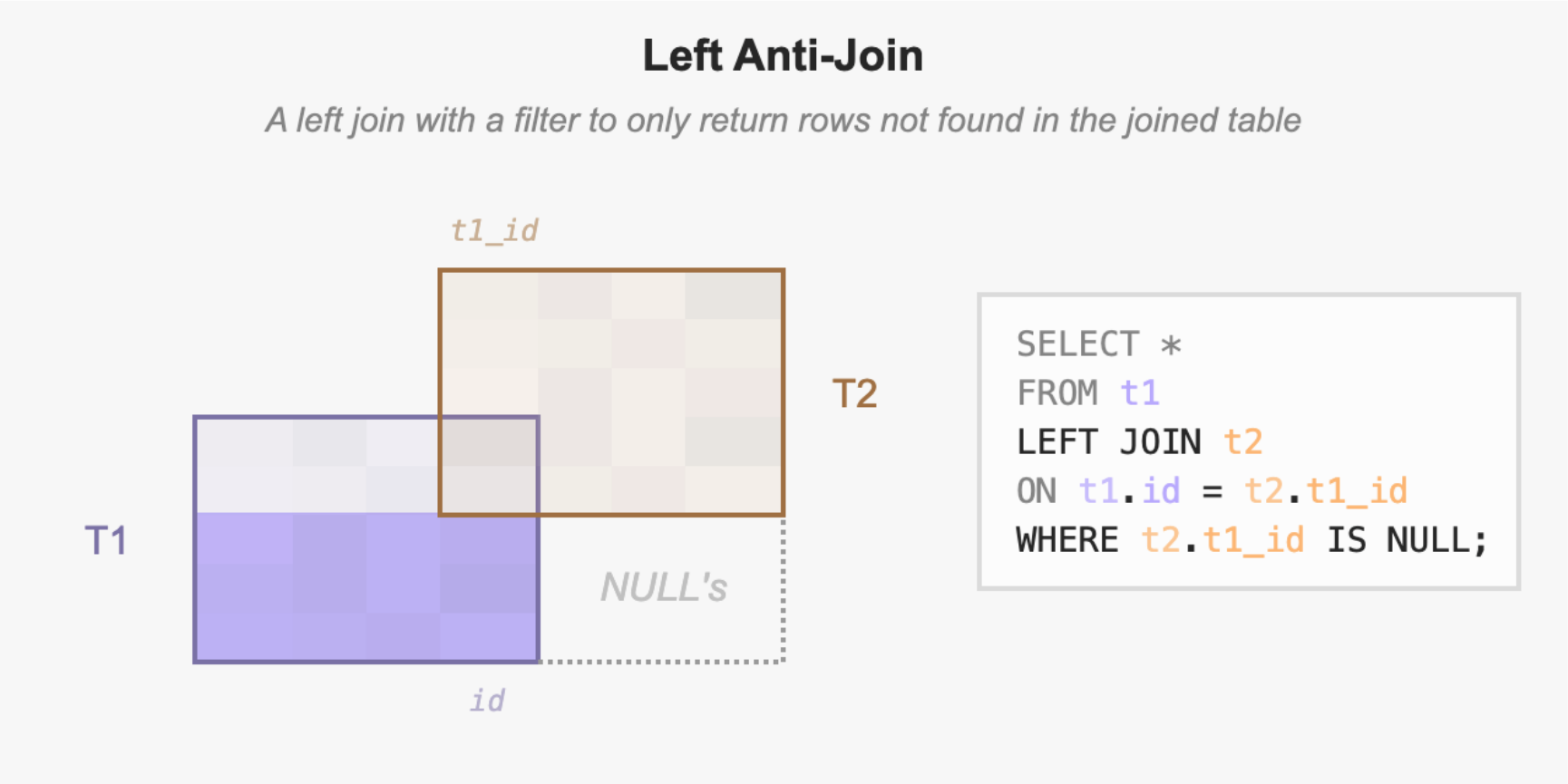
The output is:

```
team_id |  team   | city | player_id | player | role
---------+--------+------+-----------+--------+------
       3 | Bears   | Bern |      null | null   | null
       4 | Sharks | Lima |      null | null   | null
(2 rows)
```

Note that the `LEFT JOIN` is the same as the `LEFT OUTER JOIN` so you can use them interchangeably.

# LEFT ANTI JOIN

The following diagram illustrates the left join that returns rows from the left table that do not have matching rows from the right table:



# EXAMPLE JOIN QUERIES- Table creations

```
-- Cleanup for clean execution
DROP TABLE IF EXISTS lab_logs;
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS departments;

-- Create Tables
CREATE TABLE departments (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    dept_id INT REFERENCES departments(dept_id),
    enrollment_year INT NOT NULL
);
CREATE TABLE lab_logs (
    log_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    room_code VARCHAR(10),
    entry_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Insert English Data
INSERT INTO departments (dept_name) VALUES
('Industrial Engineering'),
('Systems Management'),
('Data Science'),
('Unused Department'); -- For testing RIGHT/FULL JOINs

INSERT INTO students (full_name, dept_id, enrollment_year) VALUES
('Alice Smith', 1, 2024),
('Bob Johnson', 2, 2023),
('Charlie Davis', 1, 2024),
('Diana Prince', NULL, 2025); -- Student with no department assigned

INSERT INTO lab_logs (student_id, room_code) VALUES
(1, 'LAB-A'),
(1, 'LAB-B'),
(2, 'LAB-A'),
(3, 'LAB-C');
```

# QUESTION-INNER JOIN

Task: List all students alongside their department names.

```
SELECT s.full_name, d.dept_name FROM students s INNER JOIN departments d ON
s.dept_id = d.dept_id;
```

Task: Find the names of students and the specific lab rooms they accessed, but only for students enrolled in 2024.

```
SELECT s.full_name, l.room_code FROM students s INNER JOIN lab_logs l ON
s.student_id = l.student_id WHERE s.enrollment_year = 2024;
```

Now open SQL STUDIO and answer the following question:
Write a query to find all students whose names contain the letter 'i' (case-insensitive) and show their department names. Only include students who have actually logged into a lab. Use an INNER JOIN and ILIKE.

# QUESTION-LEFT JOIN

Task: List all students and their departments. Include students who have not been assigned to a department yet.

```
SELECT s.full_name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

Task: Identify students who have **never** entered a lab. This is a common "Anti-Join" pattern.

```
SELECT s.full_name
FROM students s
LEFT JOIN lab_logs l ON s.student_id = l.student_id
WHERE l.log_id IS NULL;
```

Now open SQL STUDIO and answer the following question:
Generate a list of all departments and the count of lab entries associated with each.

Requirement 1: You must include departments that have zero lab entries.
Requirement 2: Use the COALESCE (do research on how to use this ) function to show 0 instead of NULL for departments with no activity.
Requirement 3: Ensure you are joining through the students table to reach the lab_logs.

# RIGHT JOIN

The right join is a reversed version of the left join. The right join starts selecting data from the right table. It compares each value in the `team_id` column of every row in the right table with each value in the `id` column of every row in the `teams` table.

If these values are equal, the right join creates a new row that contains columns from both tables.

In case these values are not equal, the right join also creates a new row that contains columns from both tables. However, it fills the columns in the left table with NULL.

# RIGHT JOIN

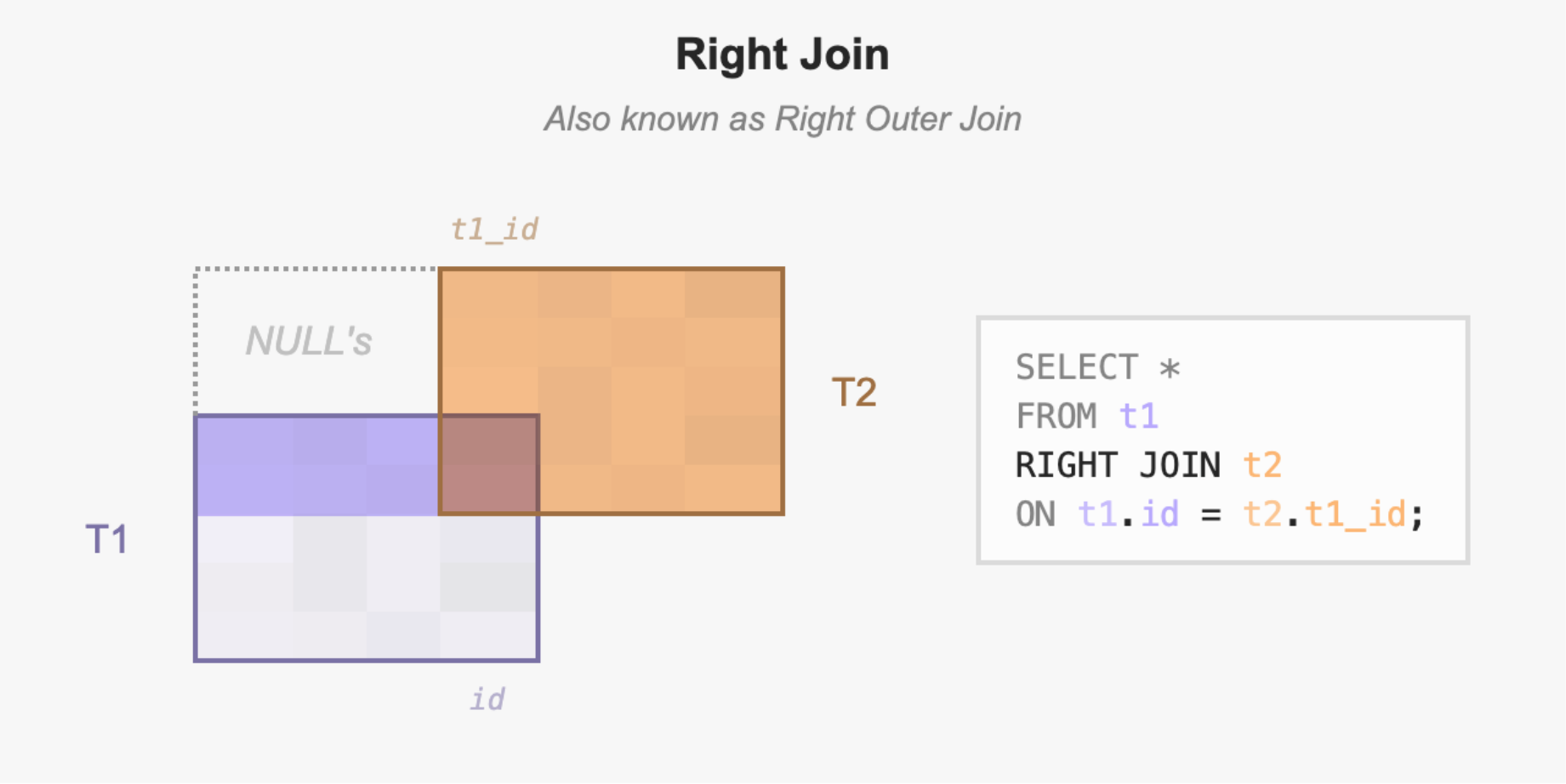
The following statement uses the right join to join the `teams` table with the `players` table:

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
RIGHT JOIN players ON teams.id = players.team_id;
```

```
team_id | team  | city | player_id | player | role
---------+-------+------+-----------+--------+-------
       1 | Lions | Rome |         1 | Ava    | Guard
       1 | Lions | Rome |         2 | Noah   | Wing
       2 | Owls  | Oslo |         3 | Emma   | Back
    null | null  | null |         4 | Liam   | Guard
    null | null  | null |         5 | Mia    | Wing
(5 rows)
```

# RIGHT JOIN

The following Venn diagram illustrates the right join:



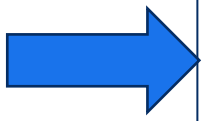

```sql
SELECT *
FROM t1
RIGHT JOIN t2
ON t1.id = t2.t1_id;
```

# RIGHT JOIN

Similarly, you can get rows from the right table that do not have matching rows from the left table by adding a `WHERE` clause as follows:

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
RIGHT JOIN players
   ON teams.id = players.team_id
WHERE teams.id IS NULL;
```

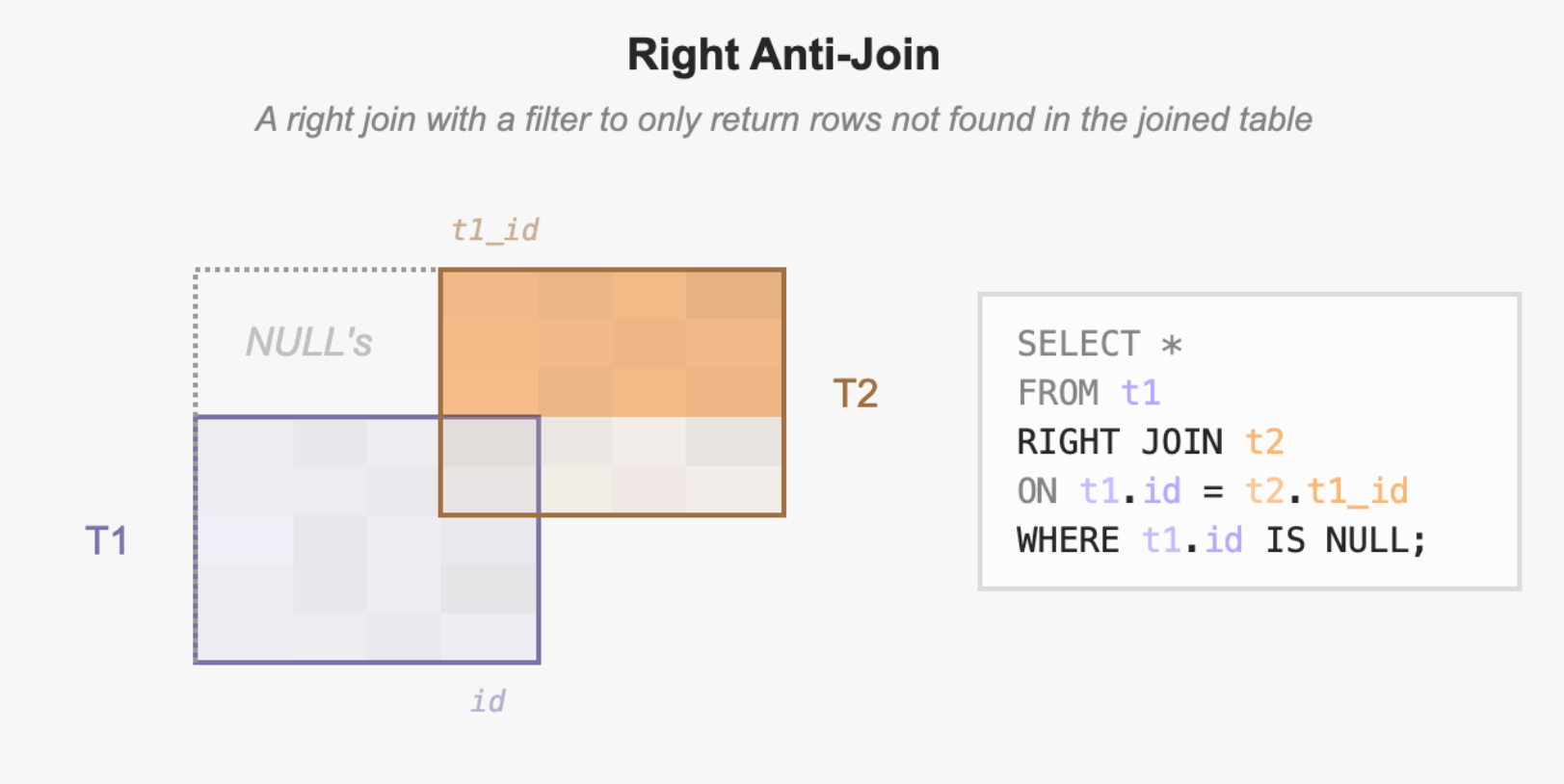
The `RIGHT JOIN` and `RIGHT OUTER JOIN` are the same therefore you can use them interchangeably.



```
team_id | team | city | player_id | player | role
---------+------+------+-----------+--------+-------
    null | null | null |         4 | Liam   | Guard
    null | null | null |         5 | Mia    | Wing
(2 rows)
```

# RIGHT ANTI JOIN

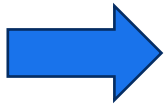
The following diagram illustrates the right join that returns rows from the right table that do not have matching rows in the left table:



# FULL OUTER JOIN

The full outer join or full join returns a result set that contains all rows from both left and right tables, with the matching rows from both sides if available. In case there is no match, the columns of the table will be filled with NULL.

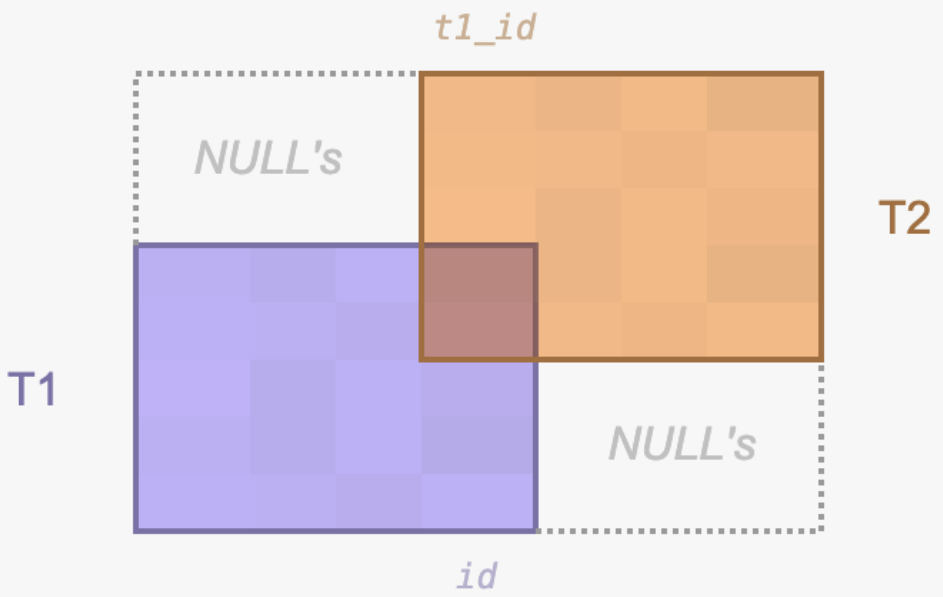
```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
FULL OUTER JOIN players
    ON teams.id = players.team_id;
```



```
team_id |  team   | city | player_id | player | role
--------+--------+------+-----------+--------+------
      1 | Lions   | Rome |         1 | Ava    | Guard
      1 | Lions   | Rome |         2 | Noah   | Wing
      2 | Owls    | Oslo |         3 | Emma   | Back
      3 | Bears   | Bern |      null | null   | null
      4 | Sharks | Lima |      null | null   | null
   null | null    | null |         4 | Liam   | Guard
   null | null    | null |         5 | Mia    | Wing
(7 rows)
```

# FULL OUTER JOIN

The following diagram illustrates the full outer join:




```sql
SELECT *
FROM t1
FULL JOIN t2
ON t1.id = t2.t1_id;
```

# FULL OUTER JOIN

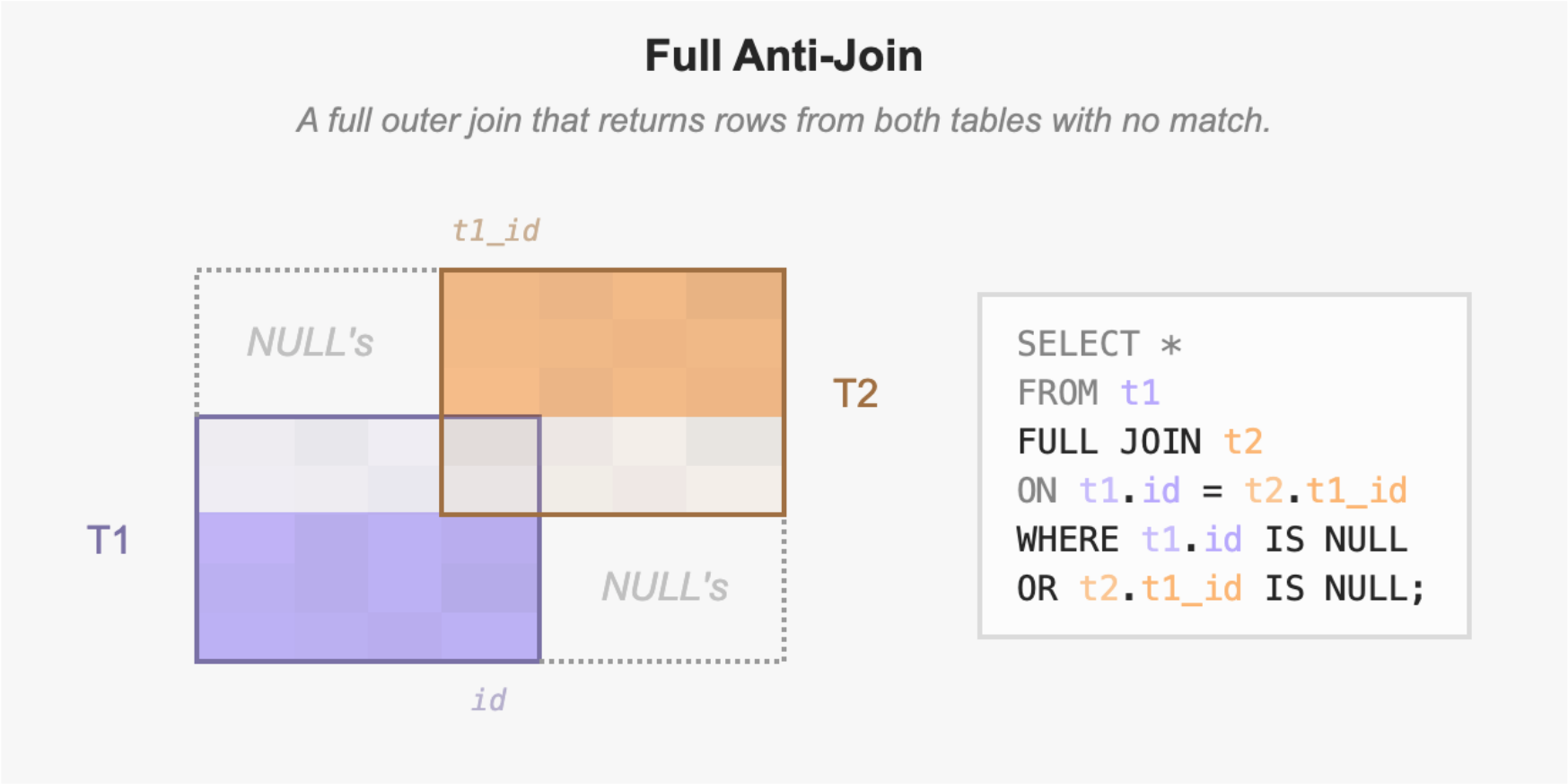
To return rows in a table that do not have matching rows in the other, you use the full join with a `WHERE` clause like this:

```
SELECT
    teams.id AS team_id,
    team,
    city,
    players.id AS player_id,
    player,
    role
FROM
    teams
FULL JOIN players
   ON teams.id = players.team_id
WHERE teams.id IS NULL OR players.id IS NULL;
```

```
team_id |  team   | city | player_id | player | role
---------+--------+------+-----------+--------+-------
       3 | Bears  | Bern |      null | null   | null
       4 | Sharks | Lima |      null | null   | null
    null | null   | null |         4 | Liam   | Guard
    null | null   | null |         5 | Mia    | Wing
(4 rows)
```

# FULL ANTI JOIN

The following Venn diagram illustrates the full outer join that returns rows from a table that do not have the corresponding rows in the other table:



# QUESTION-RIGHT JOIN

Task: List all departments and any students assigned to them. Ensure that departments with no students (like "Unused Department") still appear in the list.

```
SELECT d.dept_name, s.full_name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id;
```

Task: Find departments that currently have no students enrolled. This helps administrators identify empty programs.

```
SELECT d.dept_name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id
WHERE s.student_id IS NULL;
```

Now open SQL STUDIO and  answer the following question:
Show all lab entry timestamps and the names of the students who made them. However, prioritize the lab_logs table (Right Join) and filter for entries that occurred in 'LAB-A'. If for some reason a log exists without a valid student (an orphan record), the log should still appear.

# QUESTION-FULL OUTER JOIN

Task: List all students and all departments, matching them where possible, but showing everyone regardless of a match.

```
SELECT s.full_name, d.dept_name
FROM students s
FULL OUTER JOIN departments d ON s.dept_id = d.dept_id;
```

Task: Create a report to find "mismatches" in the system: show departments with no students AND students with no departments in a single result set.

```
SELECT s.full_name, d.dept_name
FROM students s
[redacted] departments d ON s.dept_id = d.dept_id
WHERE s.student_id [redacted] OR d.dept_id [redacted];
```

Now open SQL STUDIO and answer the following question:
Last week we discussed data integrity. Write a query using a FULL OUTER JOIN between students and lab_logs to find if there are any log entries that do not belong to an existing student (orphan records) while also listing students who have no logs. Filter the names to only show those starting with 'B' or 'D'.

# THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM